File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Python 3 (ipykernel)

Purchase Pattern Analytics(Internship Project)

Import Libraries

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load the Data

[2]:
```
df1 = pd.read_csv('Purchase Pattern Analysis(in).csv')
```

```
C:\Users\Hi\AppData\Local\Temp\ipykernel_2420\3795970726.py:1: DtypeWarning: Columns (0,4) have mixed types. Specify dtype option on import or set low_
memory=False.
  df1 = pd.read_csv('Purchase Pattern Analysis(in).csv')
```

Data Preview

[3]:
```
df1.head()
```

[3]:

	BillNo	Itemname	Quantity	Present_Date	Price	CustomerID	Country
0	536365	WHITE HANGING HEART T-LIGHT HOLDER	6.0	1/12/2010 8:26	2.55	17850.0	United Kingdom
1	536365	WHITE METAL LANTERN	6.0	1/12/2010 8:26	3.39	17850.0	United Kingdom
2	536365	CREAM CUPID HEARTS COAT HANGER	8.0	1/12/2010 8:26	2.75	17850.0	United Kingdom
3	536365	KNITTED UNION FLAG HOT WATER BOTTLE	6.0	1/12/2010 8:26	3.39	17850.0	United Kingdom
4	536365	RED WOOLLY HOTTIE WHITE HEART.	6.0	1/12/2010 8:26	3.39	17850.0	United Kingdom

[4]:
```
df1.tail()
```

[4]:

	BillNo	Itemname	Quantity	Present_Date	Price	CustomerID	Country
519329	581587	PACK OF 20 SPACEBOY NAPKINS	12.0	9/12/2011 12:50	0.85	12680.0	France
519330	581587	CHILDREN'S APRON DOLLY GIRL	6.0	9/12/2011 12:50	2.1	12680.0	France
519331	581587	CHILDRENS CUTLERY DOLLY GIRL	4.0	9/12/2011 12:50	4.15	12680.0	France
519332	581587	CHILDRENS CUTLERY CIRCUS PARADE	4.0	9/12/2011 12:50	4.15	12680.0	France
519333	581587	BAKING SET 9 PIECE RETROSPOT	3.0	9/12/2011 12:50	4.95	12680.0	France

Cleaning process : Changing Datatypes, Removing Null values

[5]:
```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 519334 entries, 0 to 519333
Data columns (total 7 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   BillNo        519334 non-null  object
 1   Itemname      517879 non-null  object
 2   Quantity      519333 non-null  float64
 3   Present_Date  519332 non-null  object
 4   Price         519333 non-null  object
 5   CustomerID    386267 non-null  float64
 6   Country       519332 non-null  object
dtypes: float64(2), object(5)
memory usage: 27.7+ MB
```

Price and Present_date column is of Objecct data type, we have to convert into float or int for price and datetime for Present_date column

[6]:

```
# 1. --- PRICE CLEANING ---
# Convert to string, remove non-numeric chars except dot, convert to float
df1['Price_clean'] = pd.to_numeric(
    df1['Price'].astype(str).str.replace('[^\d.]', '', regex=True),
    errors='coerce')

# Fill remaining NaN with 0
df1['Price_clean'] = df1['Price_clean'].fillna(0)

# 2. --- DATE CLEANING ---
# Normalize date strings: strip, replace dashes with slashes
df1['Present_Date_str'] = df1['Present_Date'].astype(str).str.strip().str.replace('-', '/')

# Convert to datetime
df1['Present_Date_clean'] = pd.to_datetime(
    df1['Present_Date_str'],
    dayfirst=True,
    errors='coerce',
    infer_datetime_format=True)

# Fill remaining NaT with a default date
df1['Present_Date_clean'] = df1['Present_Date_clean'].fillna(pd.Timestamp('2010-01-01'))

# 3. --- OTHER COLUMNS ---
# CustomerID: missing -> 'Guest'
df1['CustomerID'] = df1['CustomerID'].fillna('Guest')
```

```python
# 3. --- OTHER COLUMNS ---
# CustomerID: missing -> 'Guest'
df1['CustomerID'] = df1['CustomerID'].fillna('Guest')

# Itemname: missing -> 'Unknown'
df1['Itemname'] = df1['Itemname'].fillna('Unknown')

# Quantity: missing -> 1
df1['Quantity'] = df1['Quantity'].fillna(1)

# 4. --- DROP ORIGINAL MESSY COLUMNS ---
df1.drop(['Price', 'Present_Date', 'Present_Date_str'], axis=1, inplace=True)

# 5. --- RENAME CLEANED COLUMNS ---
df1.rename(columns={
    'Price_clean': 'Price',
    'Present_Date_clean': 'Present_Date'
}, inplace=True)

# 6. --- VALIDATION ---
print(df1.info())
print("Remaining invalid dates:", df1['Present_Date'].isna().sum())
print("Remaining invalid prices:", df1['Price'].isna().sum())
```

```
C:\Users\Hi\AppData\Local\Temp\ipykernel_2420\2984134969.py:15: UserWarning: The argument 'infer_datetime_format' is deprecated and will be removed in a future version. A strict version of it is now the default, see https://pandas.pydata.org/pdeps/0004-consistent-to-datetime-parsing.html. You can safely remove this argument.
  df1['Present_Date_clean'] = pd.to_datetime(
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 519334 entries, 0 to 519333
Data columns (total 7 columns):
 #   Column        Non-Null Count   Dtype         
---  ------        --------------   -----         
 0   BillNo        519334 non-null  object        
 1   Itemname      519334 non-null  object        
 2   Quantity      519334 non-null  float64       
 3   CustomerID    519334 non-null  object        
 4   Country       519332 non-null  object        
 5   Price         519334 non-null  float64       
 6   Present_Date  519334 non-null  datetime64[ns]
dtypes: datetime64[ns](1), float64(2), object(4)
memory usage: 27.7+ MB
```

[7]:
```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 519334 entries, 0 to 519333
Data columns (total 7 columns):
 #   Column        Non-Null Count   Dtype         
---  ------        --------------   -----         
 0   BillNo        519334 non-null  object        
 1   Itemname      519334 non-null  object        
 2   Quantity      519334 non-null  float64       
 3   CustomerID    519334 non-null  object        
 4   Country       519332 non-null  object        
 5   Price         519334 non-null  float64       
 6   Present_Date  519334 non-null  datetime64[ns]
dtypes: datetime64[ns](1), float64(2), object(4)
memory usage: 27.7+ MB
```

Descriptive Statistical Analysis

[8]:
```
df1[['Quantity','Price']].describe()
```

[8]:

	Quantity	Price
count	519334.000000	519334.000000
mean	10.104340	3.913609
std	161.529847	41.991063
min	-9600.000000	0.000000
25%	1.000000	1.250000
50%	3.000000	2.080000
75%	10.000000	4.130000
max	80995.000000	13541.330000

Minimum Quantity cannot be negetive and minimum price can't be Zero, So we have to remove the Negetive part from Quantity and Price is greater then 0(zero)

For Quantity

[9]:
```
df1[df1['Quantity'] < 0].shape
```

[9]:
```
(1334, 7)
```

[10]:
```
df1 = df1[df1['Quantity'] > 0]
```

For Price

[11]:
```
df1 = df1[df1['Price'] > 0]
```

[12]:
```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 516823 entries, 0 to 519333
Data columns (total 7 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   BillNo        516823 non-null  object
 1   Itemname      516823 non-null  object
 2   Quantity      516823 non-null  float64
 3   CustomerID    516823 non-null  object
 4   Country       516823 non-null  object
 5   Price         516823 non-null  float64
 6   Present_Date  516823 non-null  datetime64[ns]
dtypes: datetime64[ns](1), float64(2), object(4)
```

[13]:
```
df1[['Quantity' , 'Price']].describe()
```

[13]:

	Quantity	Price
count	516823.000000	516823.000000
mean	10.413706	3.932623
std	157.415113	42.092059
min	1.000000	0.001000
25%	1.000000	1.250000
50%	3.000000	2.080000
75%	10.000000	4.130000
max	80995.000000	13541.330000

Removing Duplicate records

[14]:
```
df1.duplicated().sum()
```

[14]:
```
np.int64(5251)
```

5251 duplicates records present, we have to remove duplicates otherwise it may affect Data accuracy and the Results

[15]:
```
df1.drop_duplicates(inplace = True)
```

[16]:
```
df1.info()
```

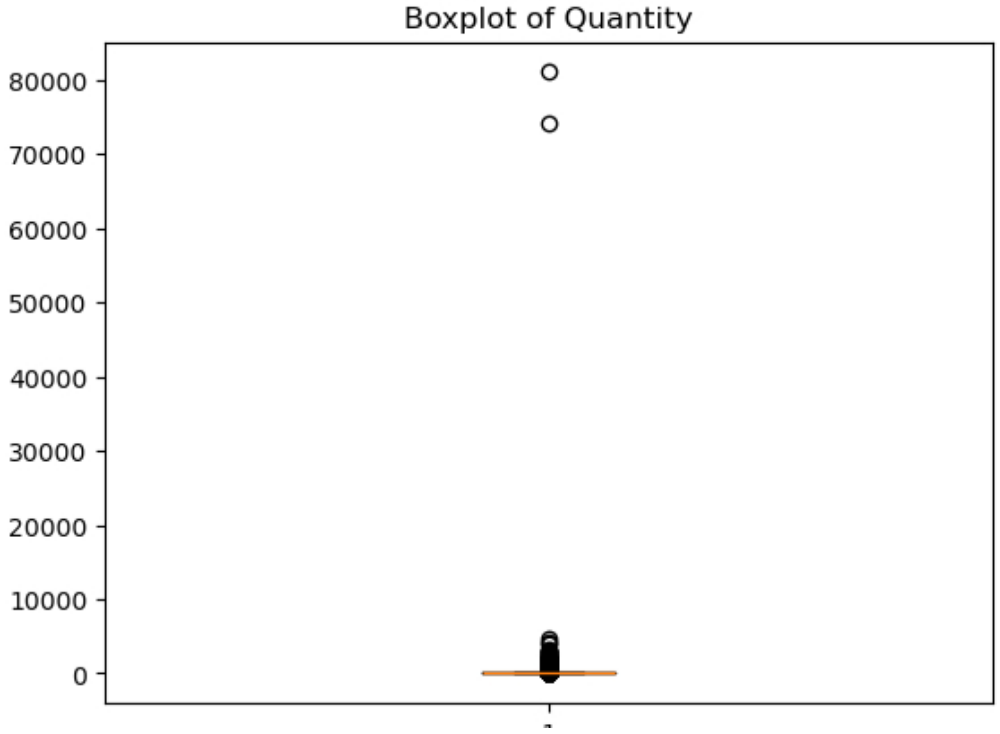
```
<class 'pandas.core.frame.DataFrame'>
Index: 511572 entries, 0 to 519333
Data columns (total 7 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   BillNo        511572 non-null  object
 1   Itemname      511572 non-null  object
 2   Quantity      511572 non-null  float64
 3   CustomerID    511572 non-null  object
 4   Country       511572 non-null  object
 5   Price         511572 non-null  float64
 6   Present_Date  511572 non-null  datetime64[ns]
dtypes: datetime64[ns](1), float64(2), object(4)
memory usage: 31.2+ MB
```

Now the is properly Cleaned and Transformed,so we can start our Visualization Part

*Visualization :

[17]:
```
### Outliers Detection(Box Plot )
```

[18]:
```
plt.boxplot(df1['Quantity'])
plt.title('Boxplot of Quantity')
plt.show()
```



It is clearly shown in BoxPlot that in Quantity there are too many Extreme Values(Outliers), so we have to Cap Quantity with some practical value like less then equal to 100

[19]:
```
df1 = df1[df1['Quantity'] <= 100]
df1.info()
```

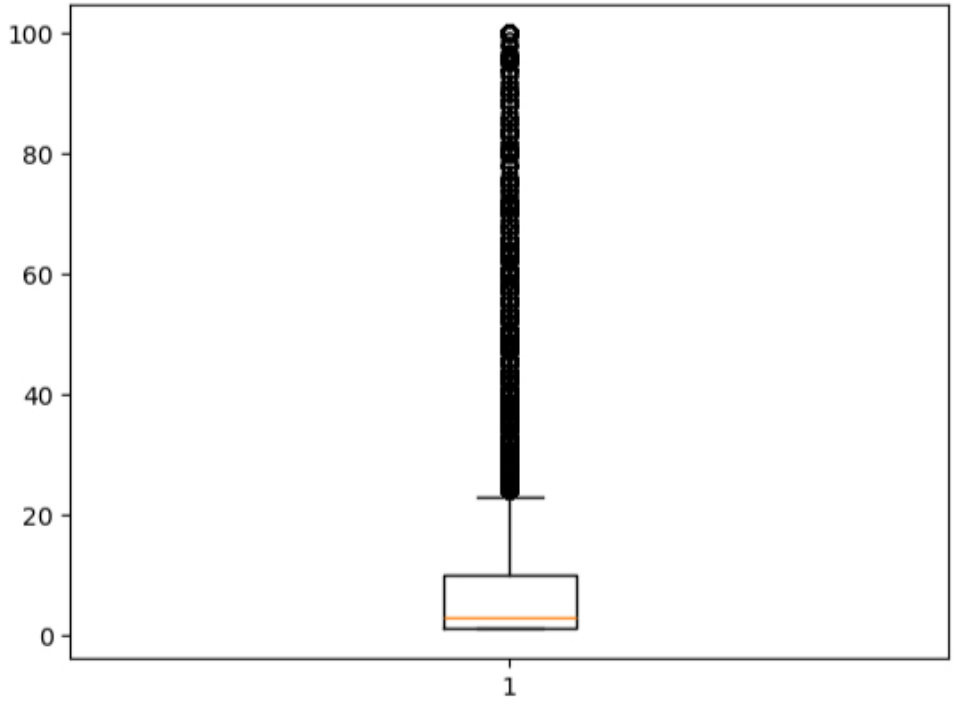
```
<class 'pandas.core.frame.DataFrame'>
Index: 506978 entries, 0 to 519333
Data columns (total 7 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   BillNo        506978 non-null  object
 1   Itemname      506978 non-null  object
 2   Quantity      506978 non-null  float64
 3   CustomerID    506978 non-null  object
 4   Country       506978 non-null  object
 5   Price         506978 non-null  float64
 6   Present_Date  506978 non-null  datetime64[ns]
dtypes: datetime64[ns](1), float64(2), object(4)
memory usage: 30.9+ MB
```

Boxplot

[20]:
```
plt.boxplot(df1['Quantity'])
plt.title('')
plt.show()
```

Boxplot

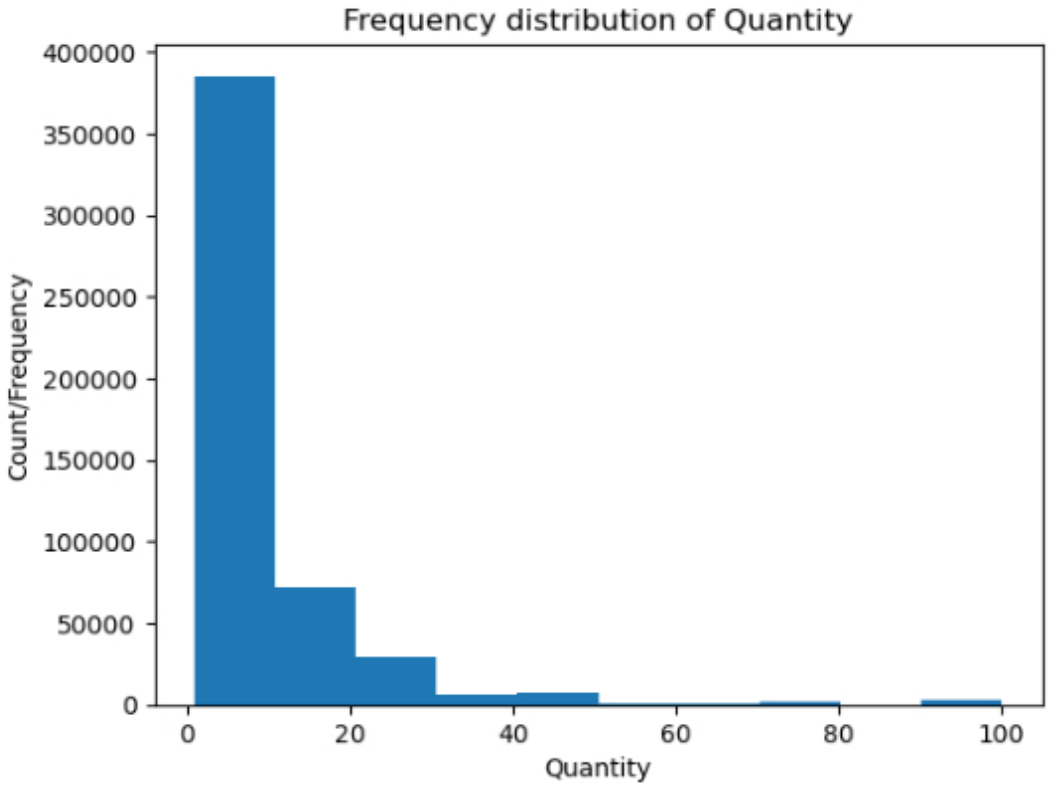
[20]:
```
plt.boxplot(df1['Quantity'])
plt.title('')
plt.show()
```



In the box plot its clearly shown that minimum quantity is 1 and the median lies between 3-4 and above whishker(lies between 21-23) are outliers

Histogram (Frequency Distribution of Quantity Column)

[21]:
```
plt.hist(df1['Quantity'] , bins = 10)
plt.ylabel("Count/Frequency")
plt.xlabel("Quantity")
plt.title("Frequency distribution of Quantity")
plt.show()
```



Insights :
- Most Purchased item quantity lies between 1 to 10 and few bulk orders which lies between 30 to 100
- Distribution is Right Skewed (Right tail is much longer)

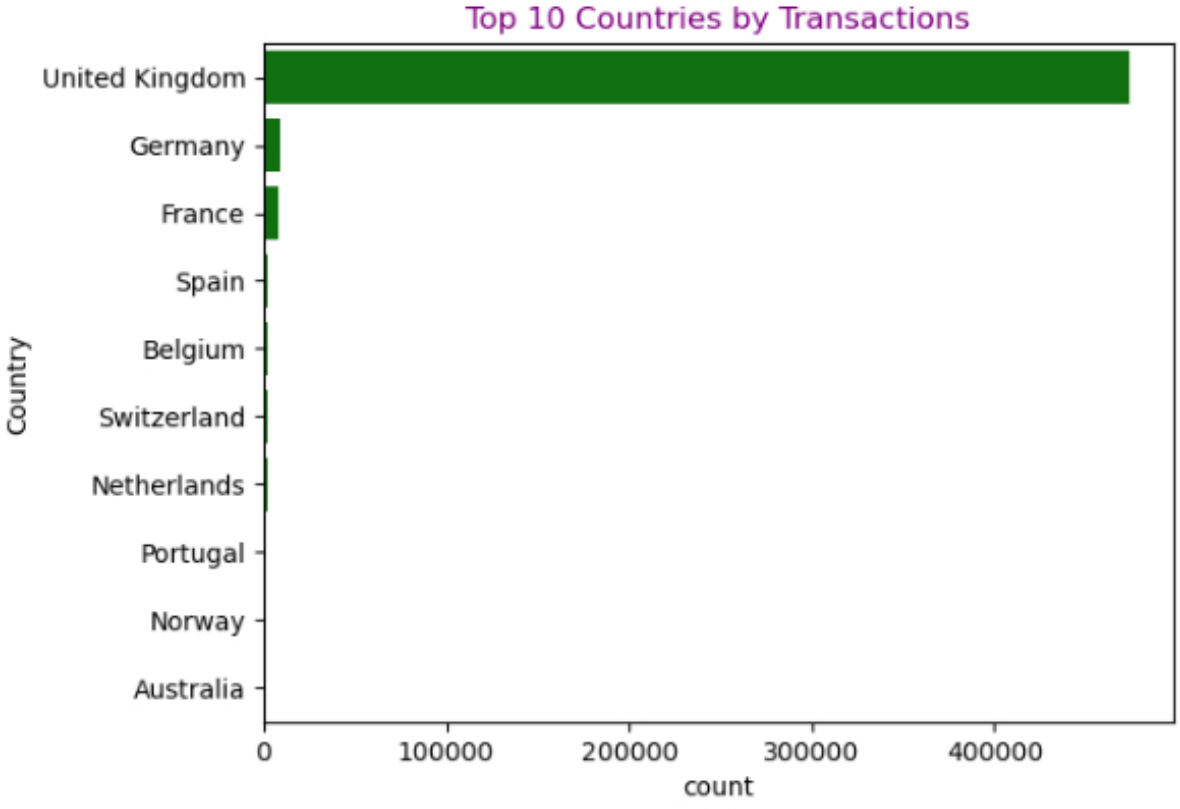
*Bar Plot :

Categorical Distribution

Country Count

[22]:
```
## top 10 countries
top10_countires = df1['Country'].value_counts().head(10).index

sns.countplot(y='Country', data = df1 , order = top10_countires , color = 'Green')
plt.title('Top 10 Countries by Transactions' , color = 'purple')
plt.show()
```



Insights :

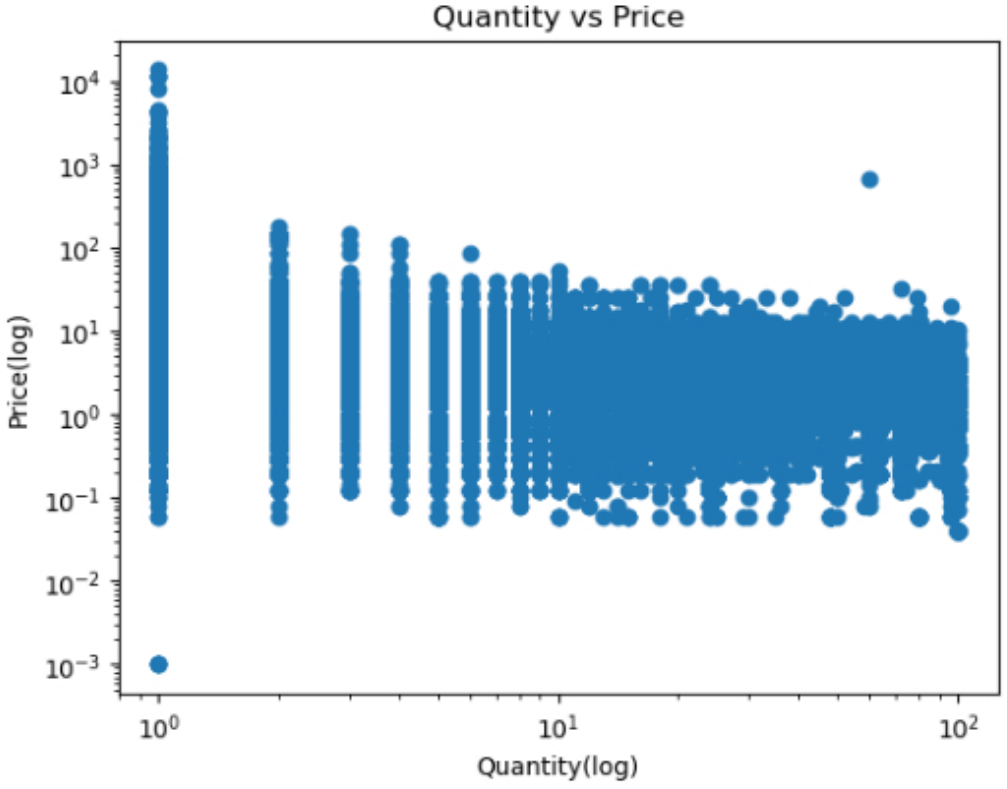
- The transactions are highly UK centric, indicating business operations are primarily focused on UK market.
- There is also a risk factor that the whole business depends on a single country market.

Scatter Plot :

- Relationship between Quantity and Price

We have to use "log transformation" on both columns as the data is mostly skewed, this helps in reducing the effect of extreme values for better plot and Insights

[23]:
```
plt.scatter(df1['Quantity'] , df1['Price'])
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Quantity(log)')
plt.ylabel('Price(log)')
plt.title('Quantity vs Price')
plt.show()
```



Insights :

- Relationship between Quantity and Price shows inverse trend, as Price increases Quantity decreases.
- Customers usually buy less quantity of item's of higher price.
- When the price falls in the low-to-medium range, customers tend to purchase higher quantities.

Time based analysis (Line-Plot)

Creating Revenue column by multiplying Quantity with Price.

[24]:
```
df1['Revenue'] = df1['Quantity'] * df1['Price']
df1['Revenue'].head()
```

[24]:
```
0    15.30
1    20.34
2    22.00
3    20.34
4    20.34
Name: Revenue, dtype: float64
```

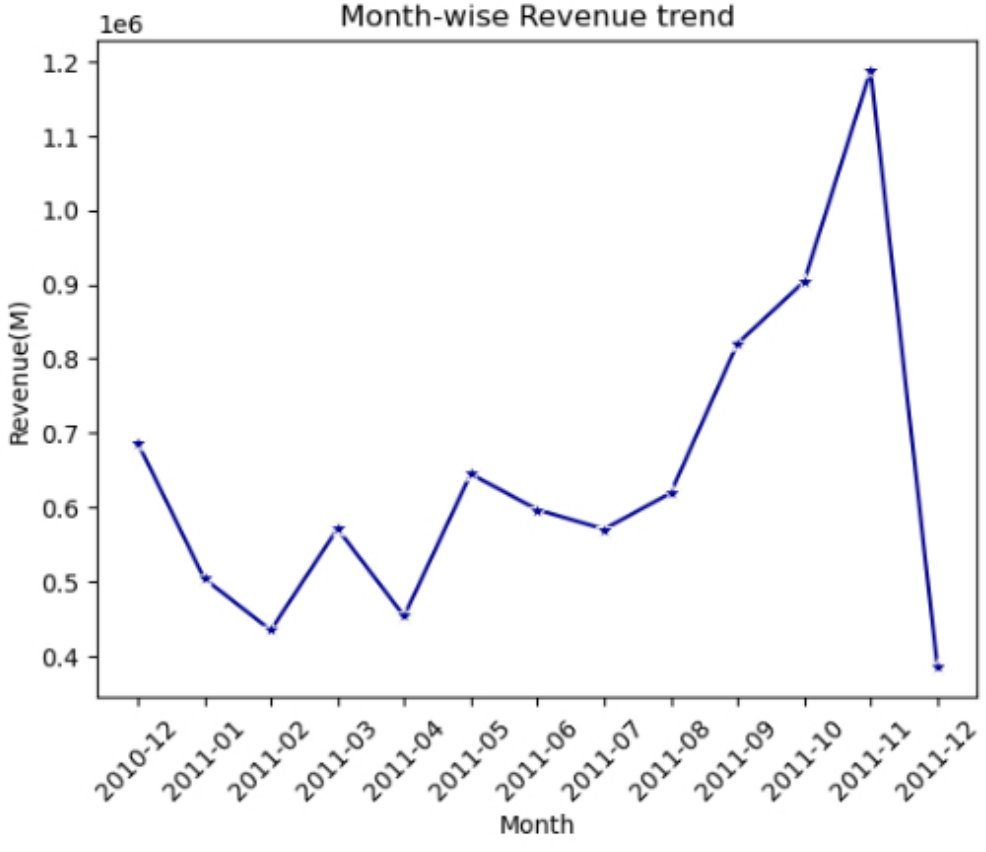
Next we have to extract Month from the Present_Date column and then Aggregate with Price to Analyze Month-wise Revenue trend

[25]:
```
df1['Month'] = df1['Present_Date'].dt.to_period('M')

monthly = df1.groupby('Month')['Revenue'].sum().reset_index()

# Change Month datatype to String
monthly['Month'] = monthly['Month'].astype(str)
```

[26]:
```
sns.lineplot(x = 'Month', y = 'Revenue', data = monthly , marker = '*' , markersize = 7, color = 'darkblue')
plt.xticks(rotation=45)
plt.ylabel('Revenue(M)')
plt.title('Month-wise Revenue trend')
plt.show()
```


Month-wise Revenue trend
1e6
1.2
1.1
1.0
0.9
0.8
0.7
0.6
0.5
0.4
Revenue(M)
2010-12
2011-01
2011-02
2011-03
2011-04
2011-05
2011-06
2011-07
2011-08
2011-09
2011-10
2011-11
2011-12
Month

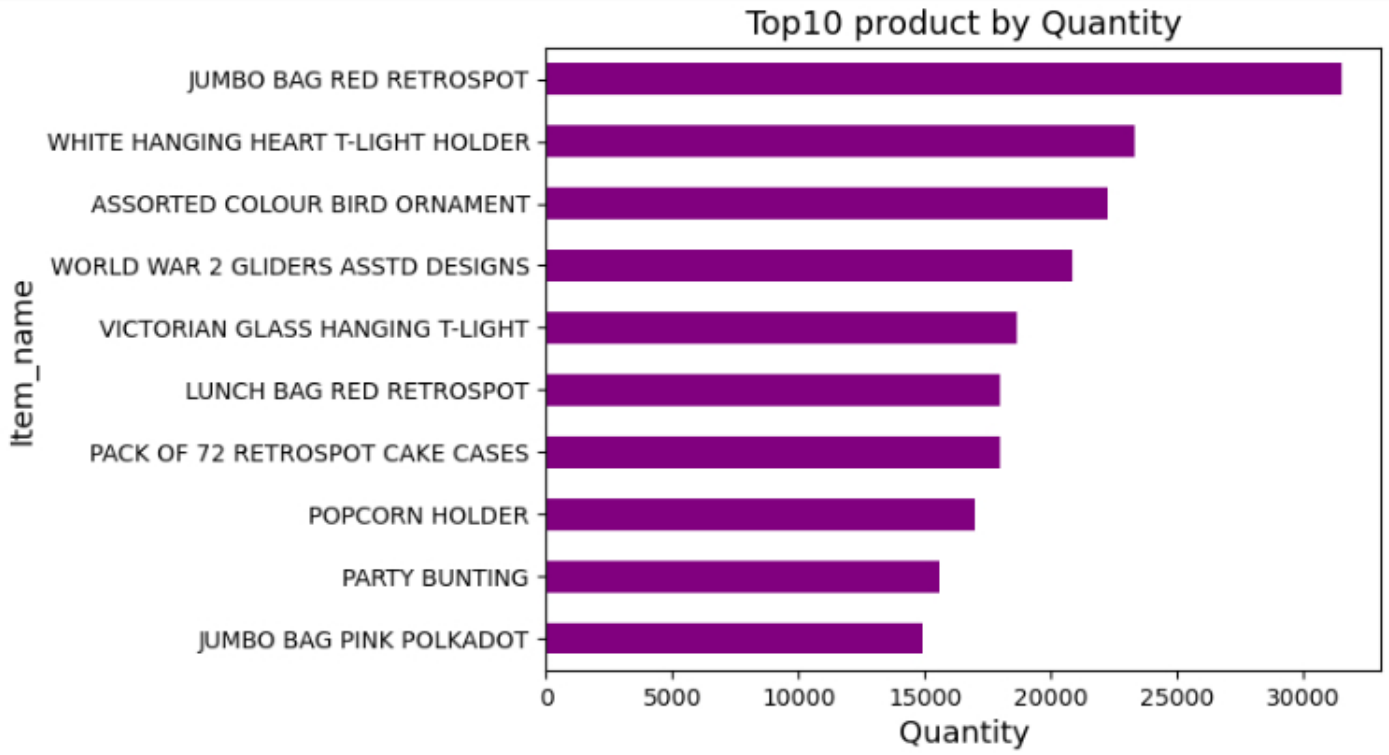
Insights

- From the month of January to May there is inconsistent customer purchasing behavior due to absence of Strong seasonal demand.
- Revenue remains relatively stable in the month of June and July.
- A clear upward growth trend from August to November, this growth corresponds to Festive and Holiday seasons.
- November records the highest revenue, indicating peak seasonal demand.
- Sharp decline in the month of December immediately after a high peak in November shows that customer already spend heavy in the festive season.

Top 10 Product by Quantity

[51]:
```
top10_product = (df1.groupby('Itemname')['Quantity'].sum().sort_values(ascending = False).head(10))

top10_product.plot(kind = 'barh' , color = 'Purple')
plt.ylabel('Item_name', fontsize = 13 )
plt.xlabel('Quantity', fontsize = 13)
plt.title('Top10 product by Quantity',fontsize = 14)
plt.gca().invert_yaxis()
plt.show()
```



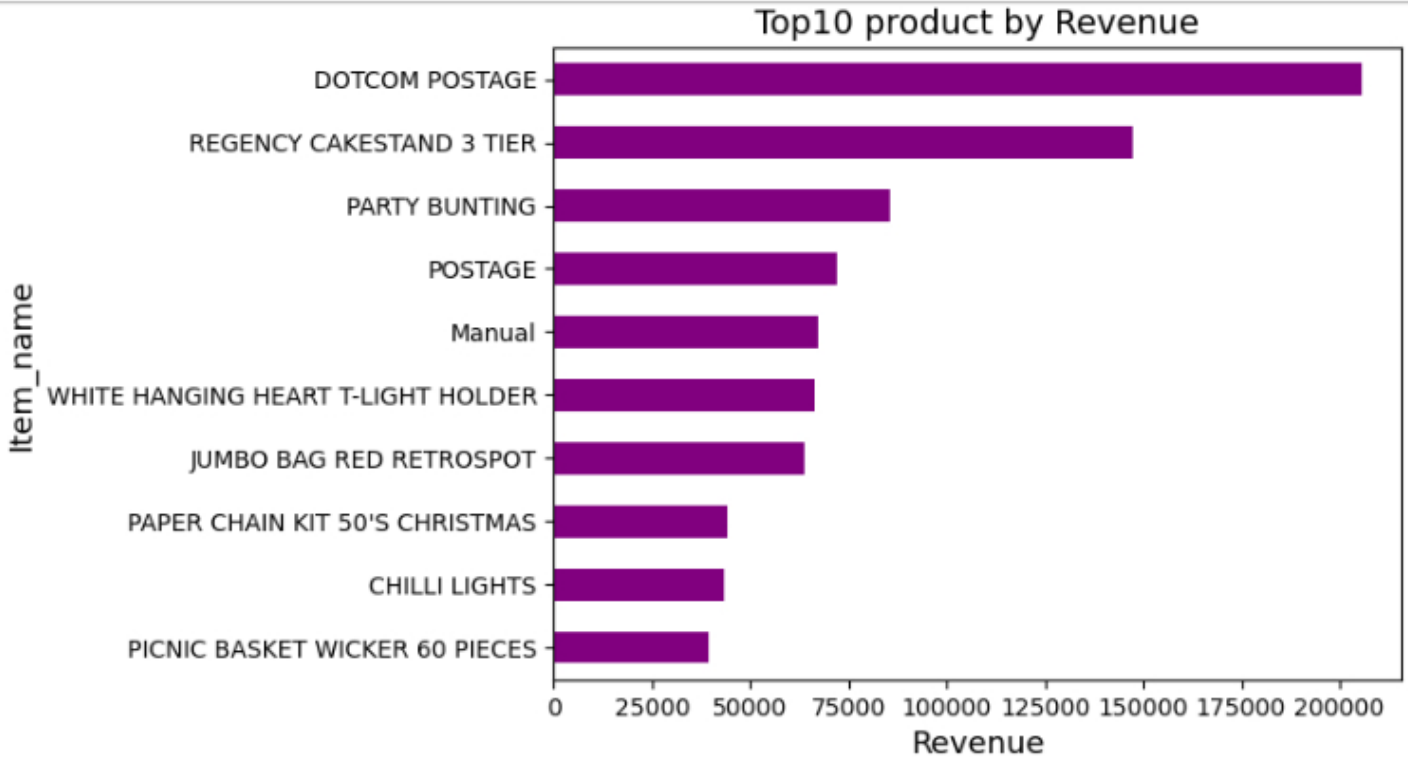
Insights

- JUMBO BAG RED RETROSPOT is the highest selling product, indicating very strong purchase demand.
- It is followed by WHITE HANGING HEART T-LIGHT HOLDER and ASSORTED COLOUR BIRD ORNAMENT.
- These top-selling products should be prioritized for inventory stocking.

Top 10 products by revenue

[54]:
```
top10_product = df1.groupby('Itemname')['Revenue'].sum().sort_values(ascending=False).head(10)

top10_product.plot(kind = 'barh', color = 'purple')
plt.xlabel('Revenue', fontsize = 13)
plt.ylabel('Item_name', fontsize = 13)
plt.title('Top10 product by Revenue',fontsize = 14)
plt.gca().invert_yaxis()
plt.show()
```



Insights

- DOTCOM POSTAGE is the highest revenue generating item followed by REGENCY CAKESTAND 3 TIER and PARTY BUNTING.
- By comparing revenue and quantity, different products rank at the top. This indicates that despite lower sales volume, certain products contribute significantly to overall revenue due to higher unit prices.